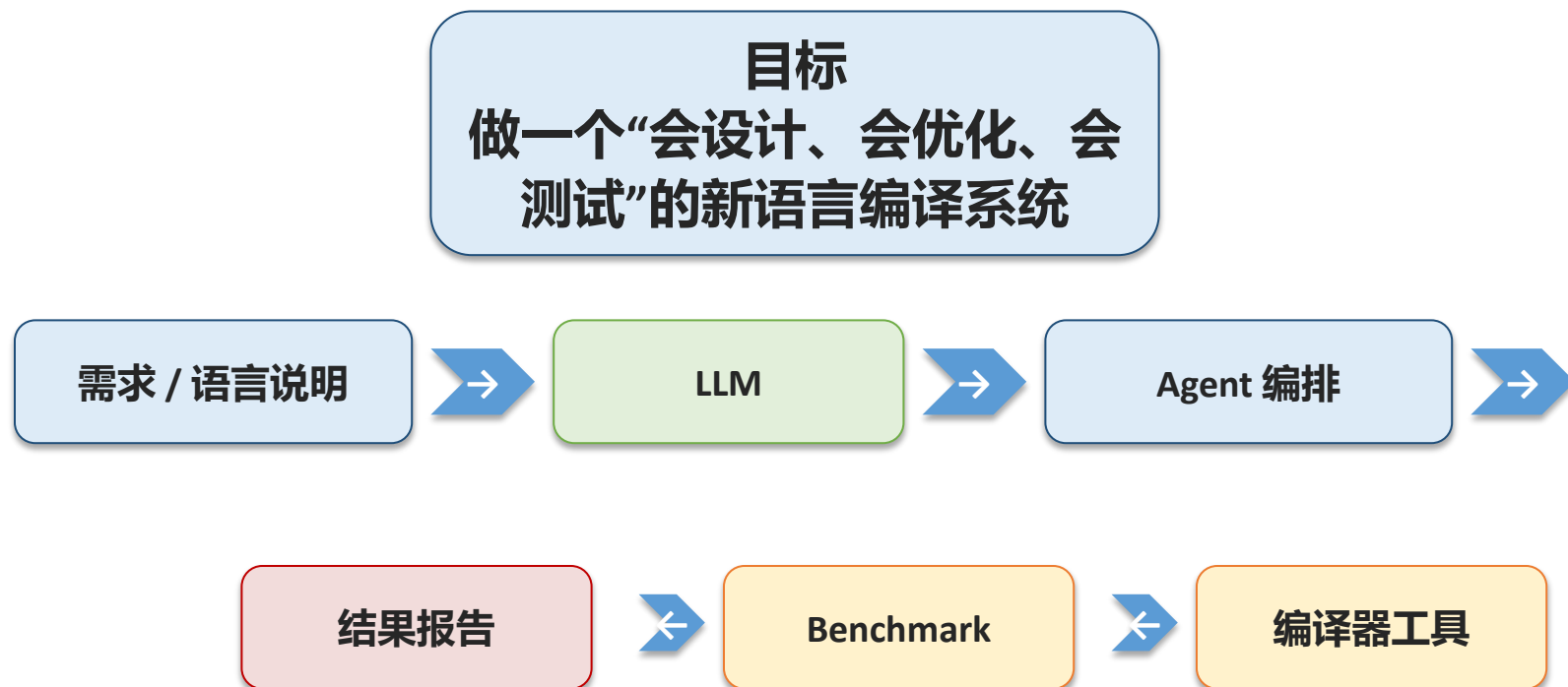


大模型、智能体驱动的面向新语言的编译系统与优化系统设计

2026.07

全哲

大模型智能体驱动的新语言编译与优化系统设计方案



做什么

不是传统编译器，而是如何用“LLM+Agent 实现编译系统功能”。重点是闭环：设计 → 执行 → 测试 → 反馈。

怎么做

先做最小新语言。
把编译、运行、测试封装成工具。
再让 LLM 和 Agent 接进去。

阶段产物 / 目标

可运行原型。
可比较的测试结果。

一句话理解：这套方案的价值，不是“模型会说”，而是“系统真的能做、能测、能改进”。

1. 先回答三个问题：做什么、怎么做、做到什么程度



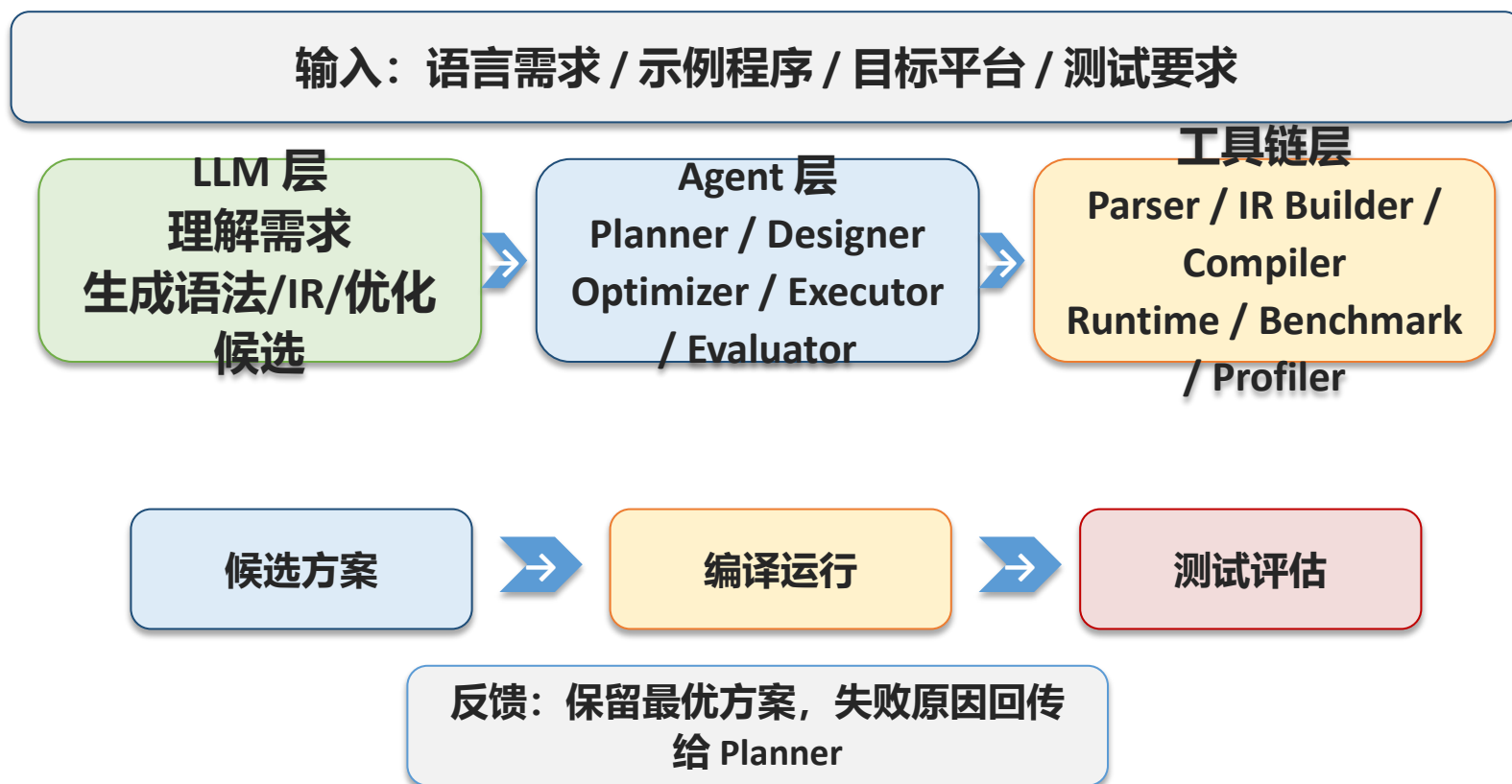
建议课堂第一分钟就讲清：最后不是交一堆图，而是交一个能跑、能测、能比较的系统。

2. 总体路线图：四个阶段推进，每个阶段都有产物



如果时间紧，只做前 3 个阶段也可以；但至少要有能运行的对比实验。

3. 总体架构图：LLM、Agent、工具链如何分工



做什么

说明三层分工。

理解 LLM、Agent、工具链各做什么。

怎么做

一切建议都要落到工具链执行

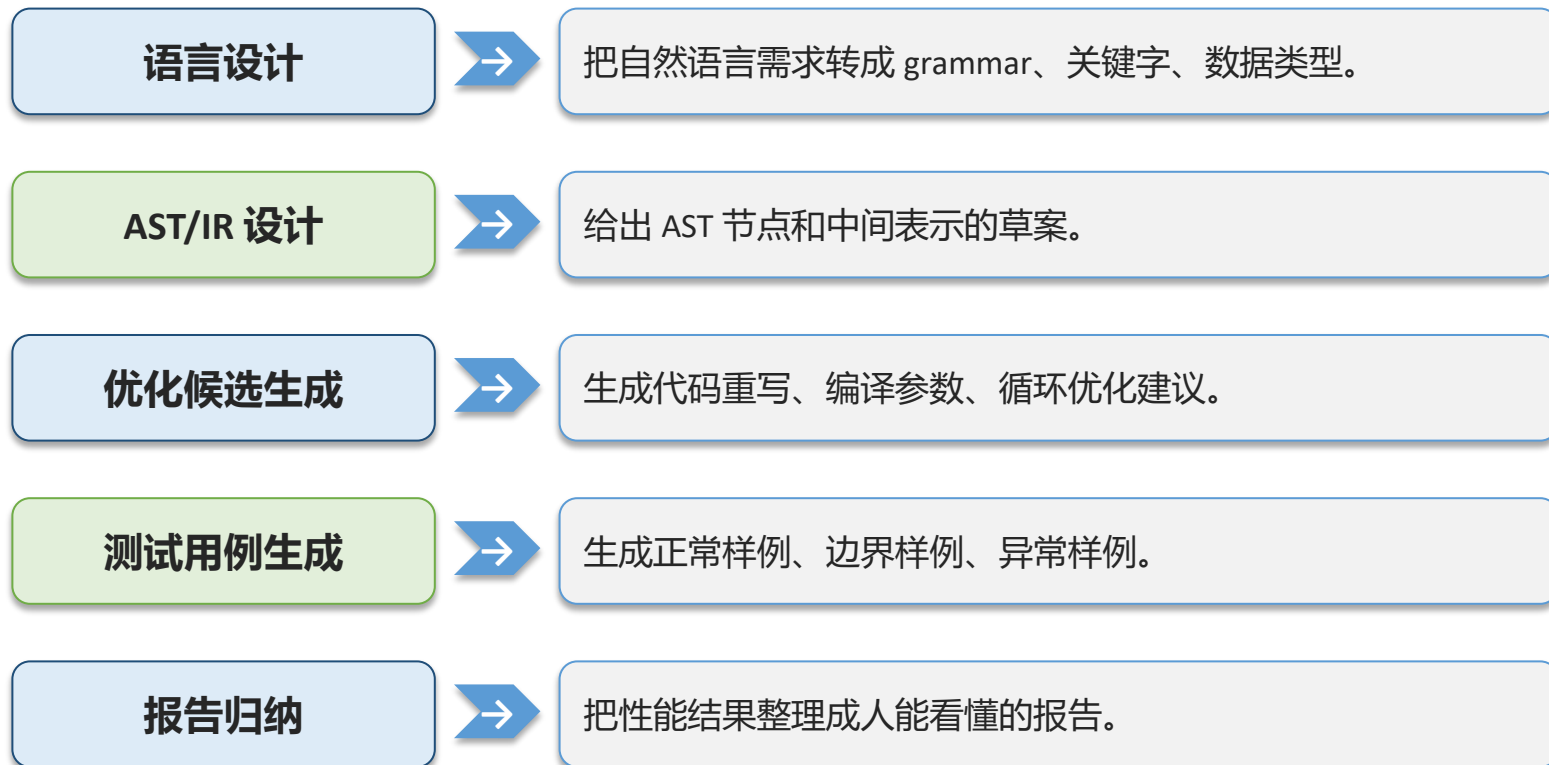
- 结果一定来自真实 benchmark

阶段产物 / 目标

总体架构图。
系统边界清晰。

最容易讲明白的一句话：LLM 负责“想”，Agent 负责“管”，工具链负责“真跑”。

4. 模块一：大模型在哪里参与？



做什么

理解：LLM 不是只会写代码。它还可以参与设计、测试、归纳。

怎么做

把每个环节需要的 Prompt 模板先设计好。
先从“给建议”开始，而不是直接让它接管全部。

阶段产物 / 目标

Prompt 模板。
设计草案。
测试样例。

本页要讲通俗：LLM 最像“会看需求、会出主意、会写草稿的助手”，不是最终裁判。

5. 模块一细化：大模型怎么用，学生要写什么

输入给 LLM 的内容

语言目标说明
几个示例程序
目标平台约束
希望优化的目标

LLM 返回的内容

语法规则草案
AST/IR 节点建议
优化候选列表
测试样例与解释

学生真正要做的事

筛选输出
接到工具链验证
记录成功/失败
逐轮改 Prompt

做什么

把“会用 LLM”变成具体动作。
要写哪些 Prompt。

怎么做

先做 3 套 Prompt。
每次实验都保存输入和输出。

阶段产物 / 目标

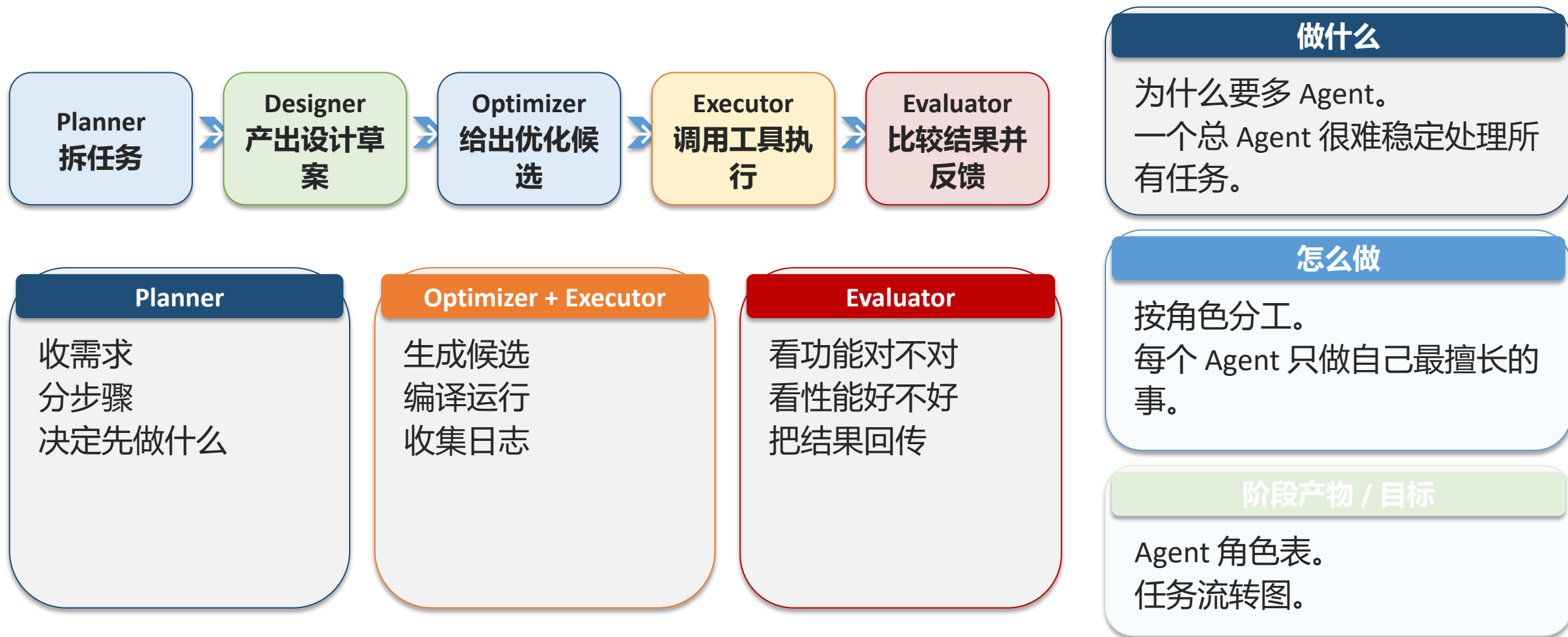
Prompt 库。
第一版 grammar / IR / 候选。

推荐最小实践

先写 3 套 Prompt：① 语言/AST/IR 设计 ② 优化候选生成 ③ 测试样例与报告生成。
每套 Prompt 都要求输出结构化结果，便于后续 Agent 调用。

阶段性产物很重要：至少要交 Prompt 文档 + 一份 LLM 输出示例。

6. 模块二：Agent 总体设计——谁来管整个流程？



课堂上可以先讲最小 3 Agent 版本：Planner + Executor + Evaluator，更容易上手。

7. 模块二细化：每个 Agent 的输入、输出、阶段产物

Agent	输入	输出	阶段产物
Planner	需求/约束	步骤计划	任务清单
Designer	语言说明	grammar/AST/IR	设计文档
Optimizer	IR/profile	候选方案	候选池
Executor	候选+工具	日志+结果	运行记录
Evaluator	日志+指标	排名+结论	对比报告

做什么

强调：每个 Agent 都要有清晰输入输出。
否则后面不好调试。

怎么做

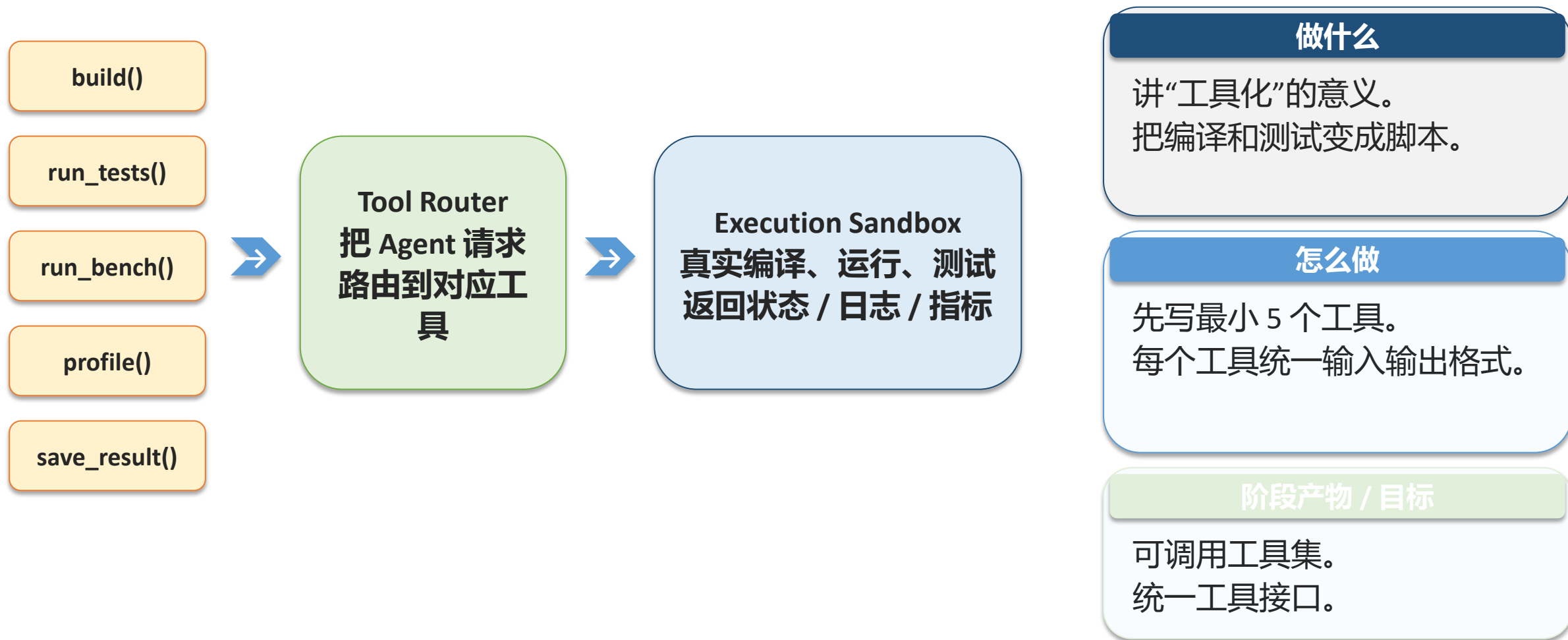
建议统一用 JSON / 表格 / 日志文件传递。
不要只传一堆自然语言。

阶段产物 / 目标

Agent 输入输出表。
可检查的阶段产物。

这页讲完，学生应该知道每个 Agent 至少要交一份什么东西。

8. 模块三：工具链怎么接？没有工具，Agent 就落不了地



没有工具调用，Agent 只能“说”；有了工具调用，Agent 才能“做”。

9. 模块三细化：优化闭环怎么跑起来？



候选可以是什么

代码重写
循环变换
编译参数
数据布局调整

评估看什么

功能是否通过
时间是否变快
内存是否变少

回传给 Planner 的信息

最好方案
失败原因
下一轮建议

做什么

解释优化闭环是整个系统的核心。
不是“生成一次就结束”。

怎么做

让候选进入真实测试。
保留最好方案，失败也要记下来。

阶段产物 / 目标

优化闭环流程图。
候选池和日志。

10. 模块四：对比测试怎么做？先有基线，再比较收益

基线版本

LLM-only

Agent-min

Agent-full

正确性				
运行时间				
吞吐量				
内存占用				
总用时				

做什么

“怎么比才公平”。
一定要有基线组。

怎么做

固定环境、固定输入、重复多次。
先过功能测试，再比较性能。

阶段产物 / 目标

对比实验表。
可重复的测试脚本。

不要只拿“最好结果”来展示，要保留完整对比表。

11. 模块四细化：测试指标与最终报告长什么样

正确性
编译成功率 / 测试通过率

性能
运行时间 / 吞吐量 / 内存

系统效率
单轮迭代时间 / 轮数

Agent 效果
成功率 / 重试率

成本
token / API / 计算开销

复现性
日志是否完整 / 脚本是否可回放

做什么

最后报告必须有什么。
不只是一张速度图。

怎么做

每次实验输出统一报告模板。
报告至少包含基线、候选、指标、结论。

阶段产物 / 目标

指标表。
统一报告模板。

推荐最小报告：程序名称、候选列表、功能测试结果、性能对比表、最优方案说明。

12. 分模块实施：学生分工建议

A 组：语言与前端

语法定义
Parser / AST
样例程序

B 组：LLM 设计

Prompt 设计
输出格式
测试样例

C 组：Agent 与工具

Planner
Executor
工具调用

D 组：测试与报告

benchmark
对比表
最终报告

推荐组织方式

每组都要有可交付产物：文档、脚本、样例、日志、结果。最后用一个整合周把四组内容接成闭环。

做什么

告诉学生如何拆分项目任务。
适合课程小组实现。

怎么做

按模块分工，最后统一集成。
每组必须交阶段产物。

阶段产物 / 目标

分工表。
集成计划。

13. 阶段一：先把“最小语言 + 最小编译链”做出来

要做什么

确定语言目标
定义 token /
grammar
生成 AST 节点
给出 basic IR
准备 5~10 个样例
程序

怎么做

LLM 先给草案
学生审查并修订
接 Parser / AST 实
现
用样例程序验证

阶段产物

语言说明文档
grammar 文件
AST/IR 定义
样例程序集
能通过样例的最小前端

做什么

把第一阶段讲透。
重点是“先能编”。

怎么做

不要急着做复杂优化。
先让最小程序能跑通。

阶段产物 / 目标

最小语言原型。
样例程序集。

第一阶段成功标准：至少有一组样例程序能从源码走到 AST/IR。

14. 阶段二：接入 LLM 与最小 Agent 闭环

要做什么

设计 Prompt
加入 Planner
加入 Executor
加入 Evaluator

怎么做

先只接 3 个 Agent
先只做 1 个任务链路
保存每次输入输出

阶段产物

Prompt 库
Agent 输入输出协议
可调用的 build/test 工具
最小闭环演示

做什么

第二阶段讲“先让系统动起来”
。别一下子上满所有 Agent。

怎么做

Planner 管流程，Executor 跑工具，Evaluator 看结果。

阶段产物 / 目标

最小 Agent 闭环。
工具调用日志。

第二阶段成功标准：能完成“生成候选 → 运行测试 → 输出结果”这条最小闭环。

15. 阶段三：形成优化闭环，进入对比实验

要做什么

生成多个优化候选
统一基线与候选对比
自动选最优
记录失败原因

怎么做

选 3~5 个程序
每个程序跑多轮
记录功能/性能/成本

阶段产物

对比实验表
最优候选列表
失败样例库
初版实验报告

做什么

第三阶段是成果最明显的一阶段。
这时能看到真实收益。

怎么做

一定要保留基线组。
不能只展示一个最好结果。

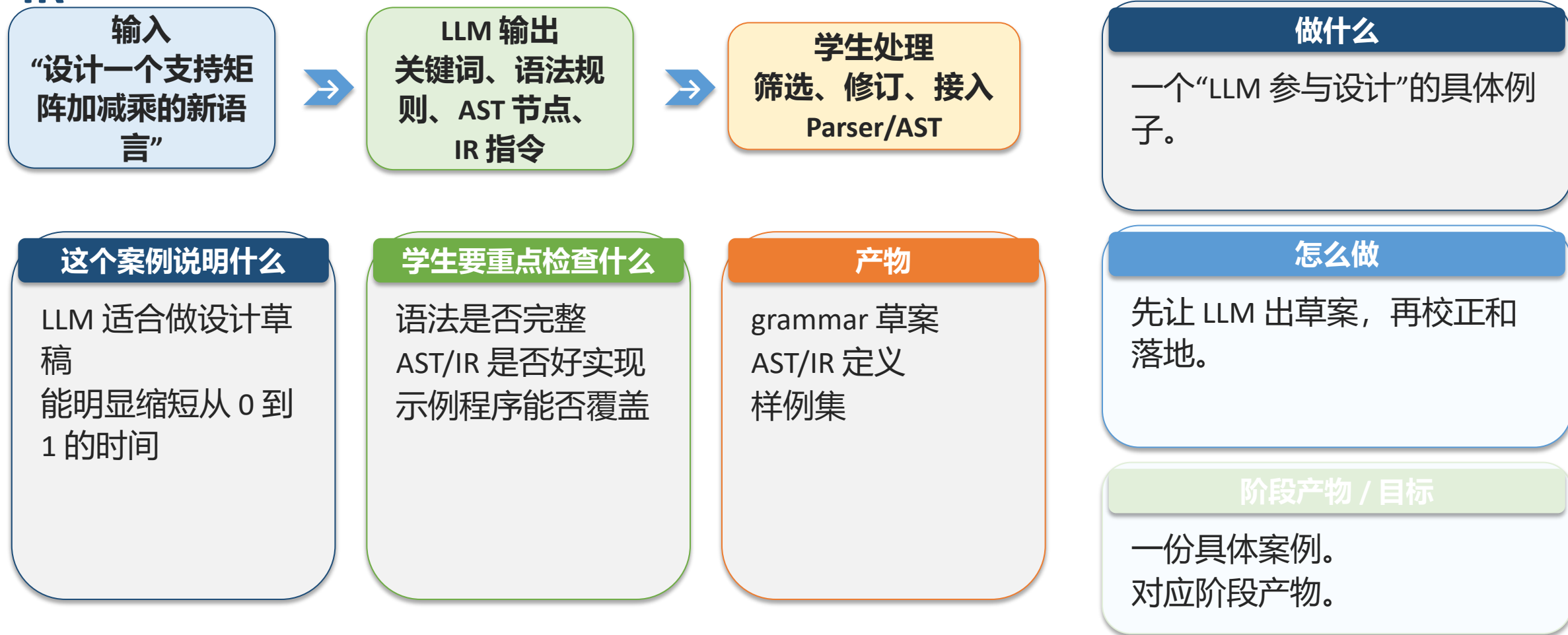
阶段产物 / 目标

对比实验结果。
最优候选结论。

第三阶段成功标准：至少能证明 1~2 个程序在正确性不变的前提下获得了明确收益。

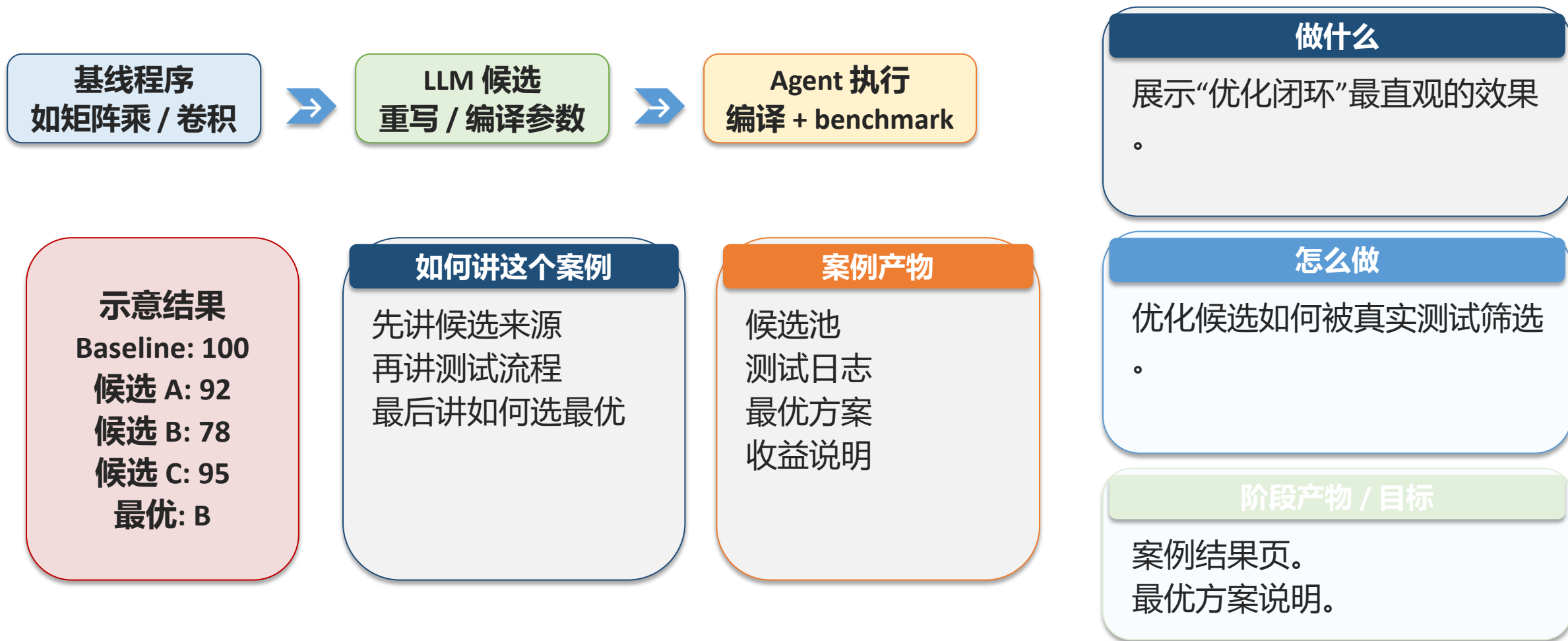
16. 案例 A：用 LLM 帮你从“语言说明”生成 grammar / AST

/ IR



案例页要尽量具体；讲“输入是什么，输出是什么，要做什么”。

17. 案例 B：针对计算内核做优化闭环



候选 B 虽然更快，也必须先确认功能测试通过。

18. 参考案例与辅助工具集（官方链接）

Agent / 编排参考

- OpenAI Agents SDK:
<https://openai.github.io/openai-agents-python/>
- OpenAI Agents Guide:
<https://developers.openai.com/api/docs/guides/agents>
- LangGraph:
<https://docs.langchain.com/oss/python/langgraph/overview>
- AutoGen:
<https://microsoft.github.io/autogen/stable/index.html>

Coding / 编译参考

- SWE-agent: <https://swe-agent.com/latest/>
- OpenHands:
<https://github.com/OpenHands/openhands>
- GitHub Copilot:
<https://github.com/features/copilot>
- LLVM: <https://llvm.org/>
- MLIR: <https://mlir.llvm.org/>

Benchmark / 工具参考

- PolyBench/C:
<https://www.cs.colostate.edu/~pouchet/software/polybench/>
- HumanEval:
<https://github.com/openai/human-eval>
- MBPP:
<https://github.com/google-research/google-research/tree/master/mbpp>
- SWE-bench:
<https://www.swebench.com/>
- google/benchmark:
<https://github.com/google/benchmark>
- hyperfine:
<https://github.com/sharkdp/hyperfine>
- pytest-benchmark:
<https://pytest-benchmark.readthedocs.io/>

做什么

可直接查阅和复现的官方资源

。

怎么做

不要只给名词，要给正确链接

。

阶段产物 / 目标

参考链接页。
工具选型依据。

建议把这页放到项目仓库 README 中，方便后续持续补充。

19. 最后一页：学生最终要交什么？

必须交付

语言说明 / grammar
Prompt 库
Agent 设计说明
工具脚本
benchmark 结果
最终报告

最小成功标准

至少一个最小语言原型
至少一个最小 Agent 闭环
至少一组对比实验
至少一个可讲清楚的案例

一句话总结

LLM 提建议
Agent 管流程
工具链真执行
Benchmark 证明
收益

做什么

把全套内容收束成“最后交什么”。

怎么做

回到项目导向，而不是只做课件。

阶段产物 / 目标

最终交付清单。
验收标准。

验收：能不能跑通、有没有对比实验、能不能说明为什么这样设计。

谢谢!

全哲

18670388716